Integrating n8n with CockroachDB

Guided Integration & Hands-on Exercises

Data Engineering & Agentic Workflows

July 28, 2026

Contents

1 Introduction

This guide provides step-by-step instructions for creating a free, cloud-hosted PostgreSQL-compatible database using CockroachDB, connecting it securely to a local n8n instance, and using database records to dynamically hydrate prompt templates for Generative AI.

2 Phase 1: CockroachDB Account & Database Setup

CockroachDB Serverless offers a generous free tier that is fully wire-compatible with PostgreSQL, making it the perfect remote database for n8n.

2.1 Step 1: Account Creation

1. Navigate to `https://cockroachlabs.cloud/signup`.
2. Create a free account using your email or GitHub/Google SSO.
3. Click **Create Cluster** and select the **Serverless (Free)** tier.
4. Choose a cloud provider (AWS/GCP) and a region closest to you.
5. Click **Create cluster**.

2.2 Step 2: Securing Connection Credentials

Once the cluster is created, a "Connection Info" modal will appear.

1. Create a new SQL user and generate a password. **Save this password immediately**, as it will not be shown again.
2. View the **Connection String** (URL format). It will look similar to:

   ```
    postgresql://username:password@free-tier-cluster-url.
   cockroachlabs.cloud:26257/defaultdb?sslmode=verify-full
   ```

3. Note down the Host URL, Port (usually 26257), Database Name (defaultdb), Username, and Password.

2.3 Step 3: Creating the Schema & Table

We need to create the table structure to hold our student records.

1. In the CockroachDB dashboard, navigate to **SQL Console** (or connect via a tool like DBeaver/pgAdmin).
2. Execute the following SQL statement to create the table:

   ```sql
   CREATE TABLE students (
       id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
       name VARCHAR(100),
       experience INT,
       assigned_track VARCHAR(10),
   ```

```
    category VARCHAR(50),
    registration_time TIMESTAMP DEFAULT current_timestamp()
);
```

3 Phase 2: n8n Credential Configuration

Because CockroachDB uses the PostgreSQL wire protocol, we use n8n's native Postgres node.

3.1 Setting up the Postgres Credential

1. Open your n8n dashboard (e.g., `http://localhost:5678` for local, or `https://n8n.io/` for subscription based).
2. On the left sidebar, click **Credentials** → **Add Credential**.
3. Search for and select **Postgres**.
4. Fill in the details exactly as extracted from your CockroachDB connection string:
    - **Host:** Your specific CockroachDB cluster URL.
    - **Database:** `defaultdb`
    - **User:** Your SQL username.
    - **Password:** Your SQL password.
    - **Port:** `26257` (Note: This is different from the default Postgres port 5432).
5. **CRITICAL STEP (SSL):** Toggle the **SSL** switch to **ON**. CockroachDB Serverless requires encrypted connections. Set the *SSL Mode* to `require` or `verify-full`.
6. Click **Save**. If configured correctly, n8n will display a "Connection Successful" message.

4 Phase 3: Saving a Record via n8n Workflow

We will now build the first part of the workflow: accepting input and inserting it into the database.

4.1 Step 1: Mock Data Input

1. Create a new n8n workflow.
2. Add a **Manual Trigger** node.
3. Add an **Edit Fields (Set)** node and connect it to the trigger.
4. Configure the Set node with the following dummy variables (representing user input or evaluated data):
    - String: `name` = `Sujeet`

- Number: `experience` = `6`
- String: `assigned_track` = `B`
- String: `category` = `Trainer`

4.2 Step 2: Database Insert Node

1. Add a **PostgreSQL** node and connect it to the Set node.
2. Select the PostgreSQL credential you created earlier.
3. Set **Operation** to **Insert**.
4. Set **Schema** to `public` and **Table** to `students`.
5. Under **Columns**, type a comma-separated list of the columns you want to map: `name, experience, assigned_track, category`.
6. Execute the node. You should see a success response showing the data inserted into CockroachDB!

5 Phase 4: Hydrating a Prompt Template

In this final phase, we will query the database and use the retrieved data to dynamically construct a prompt text (hydration) without invoking an AI agent yet.

5.1 Step 1: Fetching the Data

1. Add another **PostgreSQL** node to the workflow.
2. Set **Operation** to **Execute Query**.
3. Enter the following query to fetch the most recent record for our target user:

```sql
SELECT name, experience, assigned_track, category
FROM students
WHERE name = 'Sujeet'
ORDER BY registration_time DESC
LIMIT 1;
```

4. Execute the node to load the JSON object containing the database row.

5.2 Step 2: Prompt Hydration (Template Engine)

1. Add an **Edit Fields (Set)** node and connect it to the Query node.
2. Click **Add Value** → **String**.
3. Name the field: `hydrated_prompt`.
4. In the Value field, click the **Expression** toggle (the small gear/formula icon).

5. Write the following template using n8n expression syntax to inject the database fields dynamically:

```
System Prompt: You are a helpful educational assistant.
The current user is {{$json.name}}.
They are classified as a {{$json.category}} with {{$json.experience
   }}
years of experience. Please route all their queries strictly
   according
to Curriculum Track {{$json.assigned_track}}.
```

6. Execute the node. The output JSON will contain your final `hydrated_prompt` string, proving that data has been successfully routed from a cloud SQL database directly into an AI-ready prompt format.